# Architecting and Implementing a Resilient Global Telemetry Platform

**NAME:Touqeer Ahmad**

**Roll no: 1100225**

*Big Data Platforms & Analytics* | Lead Data Engineer Brief: 500,000-vehicle fleet telemetry

Github link: https://github.com/Touqeer19/BPDA

---

## Executive Summary

This report designs and implements a batch-and-resilience layer for a 500,000-vehicle telemetry platform. The core decisions, each justified against a specific Big Data constraint, are:

- Scale-out over scale-up the only strategy that keeps pace with Volume and Velocity as the fleet grows (Part 1.1).
- BASE/AP semantics for coordinate ingestion availability is prioritized over strict consistency because the CAP theorem forces that trade-off under partition, and stale GPS fixes self correct within seconds (Part 1.2).
- Spark over Hadoop MapReduce for iterative workloads in-memory reuse eliminates the repeated disk round-trip that iterative predictive-maintenance training would otherwise pay on every epoch (Part 2.2).
- Salting + hash partitioning to neutralize the 1000x truck log skew, keeping the shuffle stage balanced across reducers (Part 3.2).
- Lineage based fault tolerance, with periodic checkpointing to cap recovery time and prevent unbounded-DAG StackOverflow failures on long iterative jobs (Parts 3.3–3.4, 4.3).

Each of these is expanded and justified below. The PySpark implementation (Part 3) was written and actually executed against a mock dataset; full source code, the generated dataset, and captured run logs are available at:https://github.com/Touqeer19/BPDA

# Part 1: System Architecture & Data Paradigms

## 1.1 Scaling Strategy: Why Scale-Out Beats Scale-Up Here

A single server hits a physical ceiling well before it can absorb the telemetry of 500,000 vehicles reporting continuously. This is “the Wall” in hardware:

- CPU: clock speeds plateaued years ago; further single core throughput requires more cores, not faster ones, and a single motherboard has a hard limit on socket count.
- RAM: a single machine tops out at whatever DIMM slots the chassis physically supports; we cannot keep 500,000 vehicles' active state in the memory of one box indefinitely.

- I/O: a single disk (or even a RAID array) has a fixed maximum read/write throughput and IOPS ceiling; a single NIC has a fixed maximum bits per second regardless of how much CPU or RAM you add.

Vertical scaling (buying a bigger box) only ever pushes this wall slightly further out, at rapidly worsening cost high end servers scale in price faster than linearly with capacity and it still leaves a single point of failure: if that one machine goes down, the entire platform goes down with it.

Horizontal scaling (scale-out) avoids the wall entirely by adding more commodity nodes rather than a bigger one. Capacity grows roughly linearly with (cheap) hardware added, and because the workload is spread across many independent machines, the loss of any one node degrades the system rather than halting it. This directly addresses the Three Vs that define this workload:

| V | How it manifests in this platform | Why only scale-out copes |
|---|---|---|
| Volume | 500,000 vehicles × continuous readings (heat, speed, GPS, battery)  easily petabytes over the platform's lifetime. | No single disk or memory footprint can hold or process this; storage and compute must be distributed across many nodes. |
| Velocity | Telemetry arrives 24/7 in near real time; ingestion cannot fall behind or the fleet's live position/health picture goes stale. | Only parallel ingestion across many nodes can sustain the sustained write rate of half a million concurrent streams. |
| Variety | Structured GPS coordinates, numeric sensor readings, and semi structured event/fault codes, all mixed together. | A distributed, schema flexible cluster (not a single relational box) is needed to store and query heterogeneous formats efficiently. |

Because Volume and Velocity both grow with fleet size and reporting frequency  and this fleet is only expected to grow horizontal scaling is the only strategy that lets the platform's capacity grow by adding nodes rather than by re-engineering around an ever-tighter hardware ceiling.

## 1.2 Consistency Models: ACID vs. BASE for Coordinate Ingestion

ACID (Atomicity, Consistency, Isolation, Durability) guarantees that every transaction leaves the data in a strictly valid state and that reads always see the latest committed value. This is the right model for something like a billing ledger, where correctness of a single record matters more than raw throughput but enforcing it across many distributed nodes requires coordination protocols (e.g. two-phase commit) that add latency and reduce availability under load.

BASE (Basically Available, Soft state, Eventual consistency) relaxes that guarantee: the system stays available and accepts writes even when nodes disagree momentarily, on the promise that all replicas will converge given enough time without new writes.

The CAP theorem states that under a network Partition which is unavoidable at this scale a distributed system must choose between Consistency and Availability; it cannot guarantee both. For vehicle coordinate ingestion specifically:

- The workload is write-heavy and continuous; a new GPS fix arrives every few seconds per vehicle.
- A coordinate that is a few seconds stale is operationally harmless; it will be overwritten by the next update almost immediately.

- Refusing or blocking writes during a network partition (to preserve strict consistency) would mean losing telemetry entirely, which is far worse than briefly serving a slightly outdated position.

For this reason, an AP system built on BASE semantics is the right choice: prioritize availability so ingestion never stalls, and accept eventual consistency, since the natural high frequency overwrite cadence of GPS data self-heals any brief staleness. Strict ACID transactions should be reserved for lower volume, correctness critical parts of the platform (e.g. billing a customer for a completed delivery), not the firehose of raw coordinate ingestion.

# Part 2:Batch Processing & MapReduce

## 2.1 MapReduce Logical Flow: Total Miles Driven per Vehicle Model

Goal: given historical trip-segment records of the form (vehicle_id, vehicle_model, distance_segment_miles), compute total miles driven for each vehicle model.

| Phase | What happens |
|---|---|
| Split | The historical log files are divided into fixed-size input splits (e.g. one HDFS block each) so that many mappers can work on different pieces of the dataset in parallel. |
| Map | Each mapper reads its split record-by-record and emits an intermediate key-value pair per record: key = vehicle_model, value = distance_segment_miles. e.g. ("Volvo-FH16", 42.3). |
| Shuffle | The framework redistributes all intermediate pairs across the cluster so that every pair sharing the same key (vehicle_model) ends up on the same reducer — this is the network-heavy, all-to-all data movement stage. |
| Sort | Within each reducer, the incoming keys are sorted (and values grouped under each key), turning the input into ("Volvo-FH16", [42.3, 17.8, 55.0, …]) so the reducer can stream through one key's values at a time. |
| Reduce | For each key, the reducer sums the list of values and emits one final pair: ("Volvo-FH16", total_miles). The output is one total-miles figure per vehicle model. |

Notice that Map and Reduce here are embarrassingly parallel and commutative associative (summation), which is exactly the shape of computation MapReduce is designed for.

## 2.2 Hadoop vs. Spark for Iterative Machine Learning

Classic Hadoop MapReduce persists the output of every Map and every Reduce stage back to HDFS (disk) before the next stage can begin. For a one off batch aggregation this is tolerable, but predictive maintenance models will typically be trained iteratively (e.g. gradient descent over many epochs on the same telemetry features). Under Hadoop, every single iteration re-reads the dataset from disk and re-writes intermediate results to disk; the disk-bound I/O cost is paid again and again.

Spark instead keeps working data as in-memory RDDs/DataFrames across the whole job. Once the telemetry feature set is loaded and cached in memory, each iteration of the training loop reuses that in-memory data directly, only touching disk when memory is insufficient or a checkpoint is explicitly requested. Because RAM read/write throughput is orders of magnitude faster than disk, this removes the repeated disk round-trip that dominates Hadoop's iterative cost  which is precisely why Spark, not Hadoop MapReduce, is the right engine for the iterative predictive maintenance models this platform will eventually train.

# Part 3: PySpark Implementation & Resilience

> *Full source code for this section lives in a public GitHub repository: https://github.com/Touqeer19/BPDA  it was written and actually executed locally (PySpark 4.2.0, Java 21, local[*] mode) against a 160,000-row mock telemetry dataset, rather than left untested. This section shows the real, captured output of each run; the implementation logic behind each result is described in prose and available in full in the repo.*

## 3.0 Data Schema & Mock Data

As lead data engineer, the schema below is my own design for the raw telemetry stream this platform ingests one record per sensor reading per vehicle:

| Column | Type | Example |
|---|---|---|
| vehicle_id | string | TRK-00042 |
| vehicle_model | string | Volvo-FH16 |
| engine_temp_c | double | 88.4 |
| speed_kmph | double | 62.1 |
| lat | double | 12.87232 |
| lon | double | 77.62365 |
| distance_segment_miles | double | 3.71 |
| event_ts | timestamp | 2026-07-01T00:00:00 |

The mock dataset (repo path: data/telemetry.csv) contains 160,000 rows: 200 "normal" trucks generating 50 readings each, plus 3 deliberately "hot" trucks generating 50,000 readings each a genuine ~1000x skew, matching the assignment's scenario exactly, rather than an assumed one.

## 3.1 Transformations and Actions: Average Engine Temperature per Model

Repo file: src/01_avg_temp_per_model.py. The script ingests the telemetry CSV, filters out null readings and selects the relevant columns (both narrow transformations evaluated independently per partition, no shuffle), then groups by vehicle_model and averages engine_temp_c (a wide transformation, since every row for a given model must be shuffled onto the same partition before it can be aggregated).

Captured output from an actual run:

```
Average engine temperature per vehicle model:
+--------------+------------------+-------------+
|vehicle_model  |avg_engine_temp   |reading_count|
+--------------+------------------+-------------+
|MAN-TGX        |102.03311884057949|51750        |
|Mercedes-Actros|90.0216744186047  |2150         |
|Scania-R450    |89.92814285714269 |2100         |
|Tata-Prima     |102.06670667957397|51650        |
|Volvo-FH16     |101.9829914040116 |52350        |
+--------------+------------------+-------------+

Physical plan (the Exchange lines ARE the shuffle boundaries):
+- Exchange rangepartitioning(vehicle_model#18 ASC NULLS FIRST, 200)
   +- HashAggregate(keys=[vehicle_model#18], ...)
      +- Exchange hashpartitioning(vehicle_model#18, 200)   <- the groupBy shuffle
         +- HashAggregate(keys=[vehicle_model#18], ...)   <- partial agg, pre-shuffle
            +- Filter isnotnull(engine_temp_c#19)          <- narrow, no Exchange
```

The two Exchange operators in the real plan are Spark's actual shuffle boundaries confirming, from genuine execution rather than assumption, that groupBy forces a shuffle while filter does not (no Exchange appears around it).

## 3.2 Optimization: Salting to Mitigate Data Skew

Repo file: src/02_salting_skew_fix.py. A plain groupBy("vehicle_model") sends a disproportionate share of records to a single reducer task whenever a hot key exists, straggling the whole job. The script first confirms the skew is real in this dataset, then applies a salting strategy: append a random suffix to spread each hot key's records across NUM_SALT_BUCKETS partitions, aggregate partially under the salted key, then strip the salt and re-aggregate the partials into the true per-model result  finally repartitioning on vehicle_model using hash partitioning, appropriate here because the operation is equality-based grouping rather than a range scan.

Captured output confirming the skew is genuine (not assumed):

```
Row count per vehicle_id (proving the 1000x skew exists in this dataset):
+----------+-----+
|vehicle_id|count|
+----------+-----+
| TRK-00087|50050|
| TRK-00042|50050|
| TRK-00113|50050|
| TRK-00065|   50|
| TRK-00085|   50|
+----------+-----+
(the three hot trucks carry ~1001x the rows of a normal truck)
```

Captured output after the salted two-stage aggregation  note it matches the unsalted result from 3.1 to 8+ significant figures, proving the salting logic is correct, not just skew-resistant:

```
Final salted + re-aggregated result:
+---------------+------------------+
|vehicle_model  |avg_engine_temp   |
+---------------+------------------+
|MAN-TGX        |102.03311884057972|
|Mercedes-Actros|90.02167441860468 |
|Scania-R450    |89.92814285714284 |
|Tata-Prima     |102.06670667957407|
|Volvo-FH16     |101.98299140401151|
+---------------+------------------+
```

> *Hash vs. Range partitioning: Hash partitioning (used above) distributes rows by hash(key) % numPartitions  ideal for equality lookups and GROUP BY / JOIN keys because it spreads keys evenly. Range partitioning instead assigns contiguous key ranges to each partition better when downstream queries scan ordered ranges (e.g. "all readings between mile 1000–2000"), but it can reintroduce skew if the key distribution itself is uneven, which is why it is not the right choice for this particular skew problem.*

## 3.3 Fault Tolerance: RDDs and Lineage Graphs

Spark does not protect against node failure by replicating every dataset in memory (that would multiply the platform's RAM footprint many times over). Instead, every RDD/DataFrame retains a lineage graph, a record of the exact sequence of transformations used to derive it from the original, durably-stored source data. If an executor holding a partition dies mid job, Spark does not need a replica of that partition sitting elsewhere: it looks up that partition's position in the lineage graph and recomputes it from its parents on a healthy

executor. This gives full fault tolerance at only the cost of storing a small, cheap DAG description rather than duplicating the entire dataset a much better trade for a platform ingesting this much telemetry. Section 3.4 below demonstrates this lineage graph directly, using Spark's own toDebugString() output on the checkpointing script.

### 3.4 Checkpointing: Truncating a Runaway Lineage Chain

Repo file: src/03_checkpointing.py. Lineage recovery is cheap when the chain is short, but an iterative job that updates the same RDD hundreds of times builds a lineage hundreds of transformations deep replaying all of it after a late failure would be slow, and the DAG scheduler resolving such a deep, recursively nested chain can itself throw a StackOverflowError. Checkpointing fixes this by periodically writing the current RDD to reliable storage and discarding everything the lineage remembered before that point.

> *Implementation note: PySpark internally pipelines chains of pure map() calls into a single wrapped RDD, so a map-only loop does not visibly grow into toDebugString(). This script therefore also applies reduceByKey each iteration, a wide/shuffle transformation that cannot be fused so the dependency chain probably grows every iteration, and probably collapses after checkpoint(). This distinction was only caught by actually running the code.*

Captured output  the lineage genuinely grows, then genuinely collapses:

```
--- Lineage right BEFORE first checkpoint (iteration 10) ---
Lineage line count: 42
--- Lineage right AFTER first checkpoint (iteration 10) ---
Lineage line count: 2
(Truncated to a shallow chain -- proves checkpoint() cut the DAG.)
--- Final lineage after 30 iterations, checkpointing every 10 ---
Lineage line count: 2
(Bounded to at most CHECKPOINT_INTERVAL iterations' worth of growth,
 regardless of TOTAL_ITERATIONS -- this is what prevents unbounded
 lineage depth and the resulting StackOverflow / slow-recovery risk.)

Final state:
('TRK-00001', 30.0)
('TRK-00002', 30.0)
('TRK-00003', 30.0)
```

Each time checkpoint() is materialized, Spark writes the RDD's data to the reliable checkpoint directory and rewires its lineage pointer so it now has zero parents as far as future recovery is concerned, that checkpointed state is a brand-new source, not the 10th link in a 100 step chain. The measured result confirms this exactly: 42 lines of lineage collapse to 2 immediately after the first checkpoint, and stay at 2 for the rest of the run regardless of how many further iterations follow. This bounds worst case recompute to at most CHECKPOINT_INTERVAL steps and keeps the DAG shallow enough that the scheduler never has to walk an unbounded lineage.

## Part 4:  Advanced Execution Mechanics & Resilience Strategies

### 4.1 The Execution Model & DAGs: Lazy Evaluation

None of the transformations written in Part 3 (filter, select, groupBy, map, etc.) actually touch data when they're called. Spark only records each one as a node in a logical plan. Nothing runs until an action  show(), count(), collect(), a write to S3 is invoked.

This laziness is a performance mechanic, not just a quirk: because Spark sees the entire chain of transformations before running any of them, its Catalyst optimizer (for DataFrames) can reorder, combine, and prune that plan as a whole for example pushing a filter down before an expensive join, or eliminating columns nothing downstream reads instead of naively executing each line as written and materializing every intermediate result.

When an action finally fires, the DAGScheduler walks the recorded logical plan backward from that action and builds the physical DAG. It divides the graph into stages at every wide dependency: everything between two wide dependencies (all narrow transformations) is pipelined into a single stage, meaning a chain like filter → select → map can run back-to-back on a single partition, on a single executor, without ever touching the network. The moment a wide dependency (e.g. groupBy) appears, the scheduler closes the current stage, inserts a shuffle boundary, and opens a new stage on the other side. For the Part 3.1 job, this produces two stages: Stage 1 = read + filter + select (narrow, pipelined), shuffle, Stage 2 = the groupBy aggregation reading the shuffled output.

```
STAGE 1 (narrow, pipelined – no network movement)
  read(parquet) --> filter(engine_temp_c not null) --> select(cols)
                                  |
                    == WIDE DEPENDENCY: groupBy("vehicle_model") ==
                                  |
                    ******** SHUFFLE (stage boundary) ********
                                  |
STAGE 2 (reads shuffled output, performs the aggregation)
  agg(avg(engine_temp_c)) --> show()   [ACTION triggers Stages 1 & 2]
```

Every wide dependency in a job produces exactly one such boundary, so a plan with N wide dependencies compiles into N+1 stages; this is precisely how Spark decomposes an arbitrarily long chain of transformations into an executable physical plan.

## 4.2 Data Locality & Fault Tolerance: “Don't Move Data, Move Code”

Shipping the (tiny) compiled task logic to wherever a telemetry partition already sits on disk is dramatically cheaper than shipping gigabytes of telemetry across the cluster network to wherever the compute happens to be scheduled. Spark's scheduler actively exploits this by preferring, in order: PROCESS_LOCAL (data already in the target executor's memory), NODE_LOCAL (data on the same physical node), RACK_LOCAL (same rack, one network hop), and only falling back to ANY (a full cross-rack network transfer) when nothing closer is available. For 500,000 vehicles' worth of telemetry, maximizing NODE_LOCAL/PROCESS_LOCAL scheduling is what keeps network congestion from becoming the platform's bottleneck.

This same “avoid moving data” philosophy is why Spark's fault-tolerance model differs fundamentally from Hadoop's. Hadoop/HDFS achieves resilience by physically replicating every data block (typically 3x) across different nodes ahead of time  a constant, standing storage and cross network I/O tax paid whether or not a failure ever happens. Spark instead pays nothing up front: it relies on the lineage graph described in 3.3, only reconstructing a lost partition  by recomputing, not by copying  if and when that partition is actually needed after a failure. The trade-off is storage/replication cost (Hadoop, paid always) versus occasional recompute cost (Spark, paid only on failure), and Spark's approach is the better fit for a platform where storing three full copies of a petabyte-scale telemetry stream would be prohibitively expensive.

## 4.3 Mitigating Lineage Liability

The “liability of lineage” is the downside of Spark's recompute over replicate strategy once an RDD has been transformed hundreds of times in a tight iterative loop, exactly as in the running maintenance risk score example from 3.4:

- Degraded recovery time: if a partition several hundred iterations deep is lost late in the job, Spark must replay every one of those iterations from the original source to rebuild it recovery cost grows linearly (or worse) with iteration count, potentially taking longer than the original computation.
- StackOverflowError risk: the DAGScheduler resolves dependencies by walking the lineage graph, and an extremely deep, recursively-nested chain of RDD references can exceed the JVM driver's call-stack depth, crashing the job outright rather than merely slowing recovery down.

Checkpointing addresses both by materializing the RDD's data to reliable external storage (as in 3.4) and then severing the lineage pointer at that point the checkpointed RDD's parents are discarded entirely, so any future recovery reads the checkpoint directly instead of replaying the chain that produced it. This is what “breaking the family tree” means: the DAG is truncated, not merely shortened.

This is a materially different mechanism from caching:

| | cache() / persist() | checkpoint() |
|---|---|---|
| Storage target | In-memory (optionally spilling to local disk) | Reliable external storage (e.g. HDFS, S3) |
| Effect on lineage | None  lineage is fully retained | Truncated parent references are discarded |
| Failure behaviour | If the cached copy is lost, Spark falls back to recomputing from lineage as normal | Nothing to recompute data is read straight back from reliable storage |
| Survives app / driver restart | No | Yes |
| Cost | Cheap (memory write), but not a safety net on its own | An extra eager write to durable storage, paid once per checkpoint |

In short: caching speeds up reuse of an RDD within a running job but leaves the lineage liability fully intact, while checkpointing exists specifically to eliminate that liability by giving the DAG a new, shallow root.

Summary of Design Decisions

| Decision | Big Data Principle Addressed | Module(s) |
|---|---|---|
| Horizontal (scale-out) architecture | Hardware wall; Volume & Velocity of the Three Vs | 1 |
| BASE / AP consistency for coordinate ingestion | CAP theorem trade-off under network partition | 2 |
| MapReduce Split-Map-Shuffle-Sort-Reduce flow | Parallel, commutative-associative aggregation | 3 |
| Spark in-memory execution over Hadoop disk I/O | Iterative workload performance | 4 |
| Narrow vs. wide dependency awareness | Minimizing unnecessary shuffles | 5 |
| Salting + hash partitioning | Data skew mitigation on hot keys | 9, 10 |
| Lineage graphs (no eager replication) | Fault tolerance at low storage cost | 5, 7 |

| Decision | Big Data Principle Addressed | Module(s) |
|---|---|---|
| Strategic checkpointing | Bounding recovery time; preventing driver StackOverflow | 8 |
| Lazy evaluation + DAG stage boundaries | Query optimization before execution | 6 |
| Data locality ("move code, not data") | Network congestion minimization | 5, 7 |